

Systematic Hyperparameter Optimization of a Sequential Neural Network for ITSM Ticket Classification

Anthony Salva Mijares
Department of Computer Studies &
Engineering (CSE)
José Rizal University
Mandaluyong, Philippines
anthony.mijares@my.jru.edu

Abstract—This study evaluates a sequential neural network designed to dynamically classify Information Technology Service Management tickets. To establish highly controlled learning and surpass the 70.0 percent validation accuracy of the initial no-code baseline model, operations utilized a Python-based Keras environment featuring embedding layers, global average pooling, and dropout regularization. A systematic One-Factor-At-A-Time hyperparameter optimization strategy evaluated learning rates, architectural capacity constraints, and batch sizes across eleven distinct experimental configurations. The empirical findings identified the optimal model architecture, which expanded the hidden dimension to 128 neurons while maintaining a 0.005 learning rate, a value of 32 batch size, and a 0.1 dropout rate. This specific configuration successfully curbed overfitting and achieved peak predictive performance, yielding an 88.89 percent validation accuracy and a 0.8898 F1-score. Ultimately, deploying highly reliable automated triage architectures requires explicit code-based optimization to balance core network capacity with convergence stability and robust regularization

Keywords—Automated Triage, Hyperparameter Optimization, Information Technology Service Management (ITSM), Keras, Neural Networks, Text Classification.

I. INTRODUCTION & PT-P1 BASELINE PERFORMANCE

In modern enterprise Information Technology Service Management (ITSM) and IT Outsourcing (ITO) environments, service desks process continuous streams of end-user feedback and incident tickets aligned with standard ITIL frameworks. While traditional semantic analysis relies heavily on hand-coded keywords and hardcoded rule-based logic, modern neural networks construct text classifiers dynamically by converting text into numerical tokens and computing mathematical associations between word patterns and predefined labels. Initial development utilized educational platforms, such as Machine Learning for Kids, to introduce these foundational artificial intelligence concepts within a no-code web-based environment [1].

Early empirical evaluations measured how variations in training data volume and composition influenced predictive confidence across three distinct operational classes: Positive Praise, Negative Complaint, and Urgent Support. Scaling the training dataset from 30 to 180 examples substantially improved classification reliability, increasing average confidence for standard inputs from 70.2 percent to 85.0

percent, and successfully correcting sarcastic anomalies from 61.0 percent to 80.2 percent. However, testing also uncovered a critical vulnerability regarding out-of-vocabulary inputs. Unrepresented administrative edge cases were mathematically coerced into existing operational parameters with an average false-positive confidence of 96.6 percent.

These initial results confirm that deploying reliable neural networks in dynamic ITSM environments requires a combined approach of continuous, representative dataset scaling and explicit programmatic safeguards for unclassified inputs. Because visual platforms operate as closed execution systems, implementing these specific safeguards and monitoring exact mathematical convergence remains impossible. Consequently, current operations transition into an interactive, code-based notebook environment utilizing Python, TensorFlow, and Keras [2]. This shift establishes a sequential neural network capable of processing support tickets systematically while prioritizing strict hyperparameter optimization. The programmatic execution pipeline now features embedding layers, dropout regularization, and an Adam optimizer engine to establish a highly controlled learning environment. Furthermore, evaluation metrics currently extract sparse categorical cross-entropy loss curves, precision scores, and weighted F1-scores, ensuring all experimental adjustments undergo systematic logging for scientific reproducibility [3].

II. EXPERIMENTAL METHODOLOGY & HYPERPARAMETER CONFIGURATIONS

To evaluate the sequential neural network classifier for IT support ticket classification, a systematic hyperparameter optimization strategy was implemented. The experimental framework tracks structural modifications across learning rates, hidden dimensions, dropout rates, epoch caps, and batch sizes, with all configuration parameters.

TABLE I. EXPERIMENTAL HYPERPARAMETER CONFIGURATIONS

Run ID	Hyperparameter Configuration & Strategy					
	Learning Rate	Hidden Dim	Drop out Rate	Epochs	Batch Size	Objective
PT-P1-BASE LINE	N/A	N/A	N/A	N/A	N/A	Raw baseline metrics from ML

Run ID	Hyperparameter Configuration & Strategy					
	Learning Rate	Hidden Dim	Drop out Rate	Epochs	Batch Size	Objective
						for Kids
EXP-01	0.005	64	0.1	15	32	Initial Keras Baseline
EXP-02	0.005	64	0.1	10	32	Epoch Optimization
EXP-03	0.001	64	0.1	10	32	Isolate Lower Learning Rate
EXP-04	0.01	64	0.1	10	32	Isolate Higher Learning Rate
EXP-05	0.005	32	0.1	10	32	Isolate Reduced Hidden Dimension
EXP-06	0.005	128	0.1	10	32	Isolate Expanded Hidden Dimension
EXP-07	0.005	128	0.0	10	32	Isolate Dropout Rate (Test No Regularization 0.0)
EXP-08	0.005	128	0.3	10	32	Isolate Dropout Rate (Test Moderate Regularization 0.3)
EXP-09	0.005	128	0.5	10	32	Isolate Dropout Rate (Test Heavy Regularization 0.5)
EXP-10	0.005	128	0.1	10	16	Isolate Batch Size (Test Smaller Batch Size 16)
EXP-11	0.005	128	0.1	10	64	Isolate Batch Size (Test Larger Batch Size 64)

As detailed in Table I, the experimental evaluation follows a systematic One-Factor-At-A-Time hyperparameter optimization strategy to evaluate the sequential neural network classifier. The evaluation workflow begins with PT-P1-BASELINE, which utilizes raw baseline metrics from ML for Kids to establish an initial performance reference

point. EXP-01 then establishes the initial Keras baseline using a learning rate of 0.005, a hidden dim of 64, a dropout rate of 0.1, a 15-epoch training window, and a batch size of 32. To prevent the late-stage divergence observed in the initial baseline, EXP-02 restricts the training duration to 10 epochs while keeping the learning rate at 0.005, hidden dim at 64, dropout rate at 0.1, and batch size at 32 constant. Building upon this configuration, EXP-03 and EXP-04 evaluate learning rate sensitivity by modifying the parameter to 0.001 to target slower weight updates and 0.01 to target rapid early learning dynamics, respectively, while holding hidden dim at 64, dropout at 0.1, epochs at 10, and batch size at 32. Architectural capacity constraints are subsequently tested in EXP-05 and EXP-06 by scaling the hidden dim down to 32 and up to 128 using a 0.005 learning rate, 0.1 dropout rate, 10 epochs, and a batch size of 32. Furthermore, regularization effects are analyzed in EXP-07, EXP-08, and EXP-09 by varying the dropout rate to 0.0 for zero regularization, 0.3 for moderate regularization, and 0.5 for heavy regularization on an expanded hidden dim of 128, while maintaining a 0.005 learning rate, 10 epochs, and a batch size of 32. Finally, EXP-10 and EXP-11 inspect stochastic gradient variance and update stability by adjusting the batch size to 16 and 64, respectively, while preserving a 0.005 learning rate, a hidden dim of 128, 0.1 dropout rate, and 10 epochs.

III. PERFORMANCE MONITORING & LOSS CURVE ANALYSIS

Performance monitoring and loss curve analyses were conducted across all experiments to evaluate convergence behavior, generalization, and stability of the sequential neural network. The evaluation starts with PT-P1-BASELINE. The initial phase uses raw metrics from the ML for Kids platform to set a fundamental performance reference for the IT support ticket classification prior to the systematic evaluation of the sequential neural network.

The detailed metrics for the subsequent experiments show the interaction between training loss and validation loss. The resulting data demonstrates how changing hyperparameters, such as learning rate, hidden dim, dropout regularization, and batch size, directly affects model optimization.

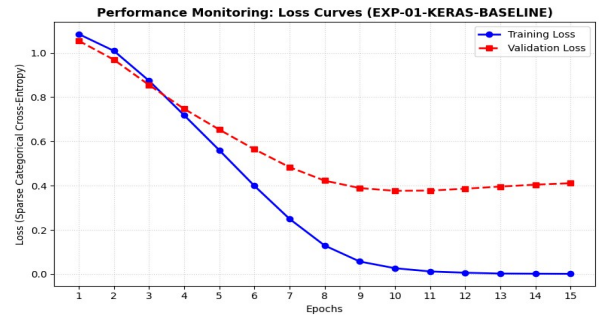


Fig. 1. Initial Keras baseline capped at 15 epochs (EXP-01).

As shown in Fig. 1, the model achieves a training loss of 0.0028 and a validation loss of 0.3586, with a training accuracy of 100.00% and a validation accuracy of 81.50% (precision: 0.8407, recall: 0.8148, F1-score: 0.8179). The curves show a rapid decrease in training loss and a stable validation loss minimum at around epoch 10, before minor divergence at the end.

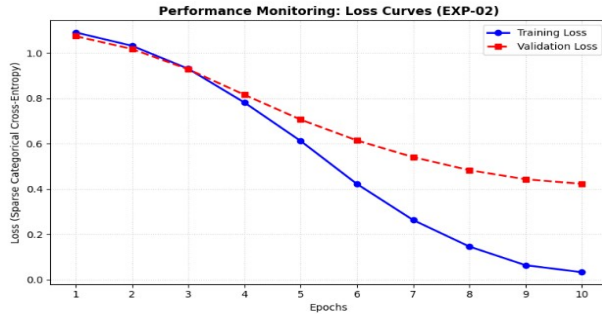


Fig. 2. Epoch optimization (EXP-02).

Illustrated in Fig. 2, limiting training to 10 epochs yields a training loss of 0.0327, a validation loss of 0.4225, 100.00% training accuracy, and an improved validation accuracy of 85.20% (precision: 0.8770, recall: 0.8519, F1-score: 0.8532). The 10-epoch configuration successfully prevents the late-stage divergence seen in the baseline.

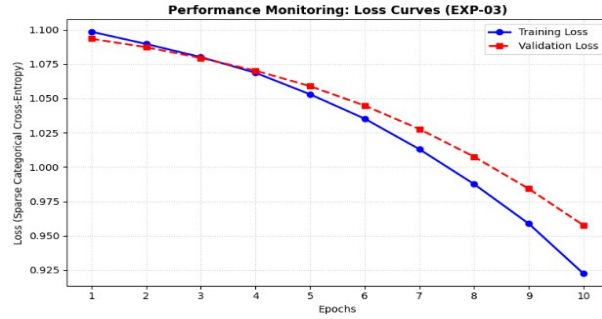


Fig. 3. Lower learning rate of 0.001 (EXP-03).

As stated in Fig. 3, the run records a high training loss of 0.9224 and a validation loss of 0.9576, with 98.41% training accuracy, 81.50% validation accuracy, a precision of 0.8330, recall of 0.8148, and an F1-score of 0.8179. The curves show slow convergence and underfitting, with very little loss reduction for both training and validation.

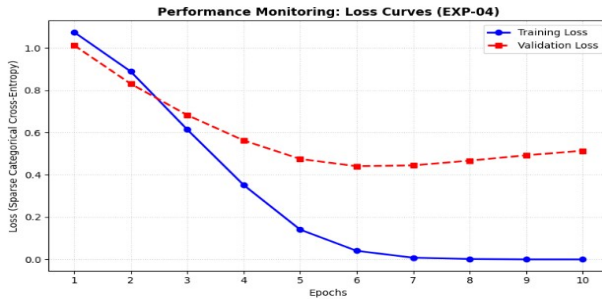


Fig. 4. Higher learning rate of 0.01 (EXP-04).

As shown in Fig. 4, the model produces a training loss of 0.0003, a validation loss of 0.5134, 100.00% training accuracy, 85.19% validation accuracy, a precision of 0.8770, recall of 0.8519, and an F1-score of 0.8643. The higher learning rate makes early learning faster, but causes significant validation loss divergence after epoch 6.

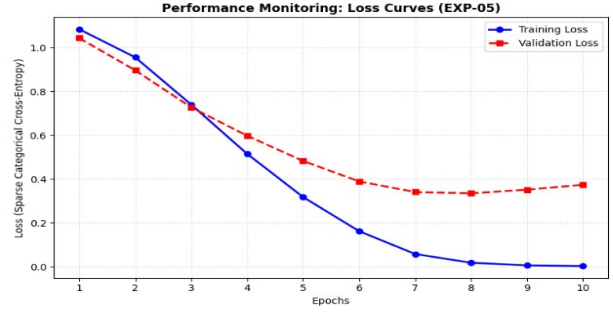


Fig. 5. Reduced hidden dim of 32 (EXP-05).

Illustrated in Fig. 5, compressing the network capacity records a training loss of 0.0022, a validation loss of 0.3736, 100.00% training accuracy, 85.19% validation accuracy, a precision of 0.8333, recall of 0.8148, and an F1-score of 0.8179. The curves show a slight change in the loss trend during training and validation because of the limited network capacity.

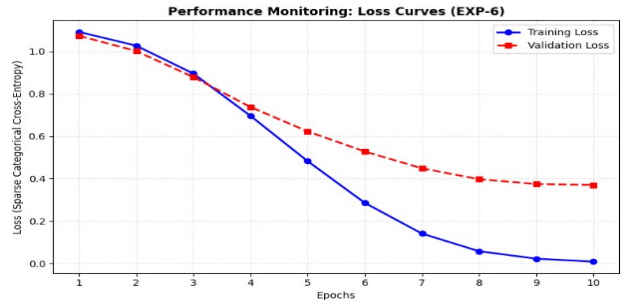


Fig. 6. Expanded hidden dim of 128 (EXP-06).

As depicted in Fig. 6, increasing the hidden dim achieves peak performance with a training loss of 0.0086, a validation loss of 0.3702, 100.00% training accuracy, an 88.89% validation accuracy, a precision of 0.9016, recall of 0.8889, and a top F1-score of 0.8898. The graph shows a smooth, consistently decreasing validation curve, indicating better feature representation.

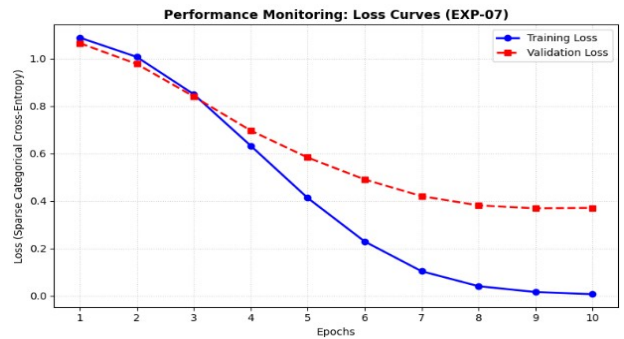


Fig. 7. Zero dropout regularization (EXP-07).

As shown in Fig. 7, removing dropout completely from the model with a hidden dim of 128 results in a training loss of 0.0067, a validation loss of 0.3705, 100.00% training accuracy, 81.48% validation accuracy, a precision of 0.8333, recall of 0.8148, and an F1-score of 0.8170. The lack of dropout widens the generalization gap and increases the divergence between training loss and validation loss.

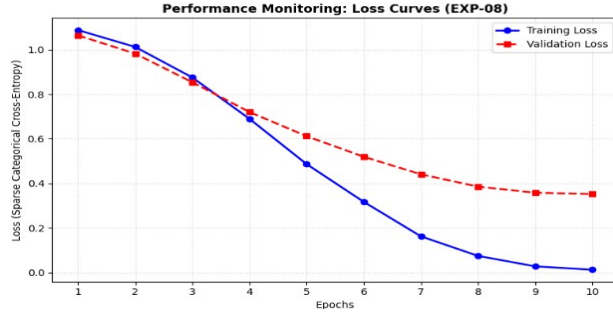


Fig. 8. Moderate dropout rate of 0.3 (EXP-08).

Illustrated in Fig. 8, using moderate regularization yields a training loss of 0.0127, a validation loss of 0.3522, 100.00% training accuracy, 85.19% validation accuracy, a precision of 0.8611, recall of 0.8519, and an F1-score of 0.8545. The graph demonstrates dropout effectively smoothing the validation loss curve and controlling variance.

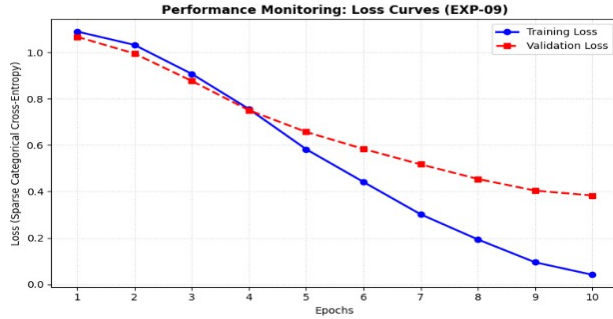


Fig. 9. Heavy dropout rate of 0.5 (EXP-09).

As depicted in Fig. 9, heavy regularization results in a training loss of 0.0408, a validation loss of 0.3826, 100.00% training accuracy, 81.48% validation accuracy, a precision of 0.8321, recall of 0.8148, and an F1-score of 0.8234. The loss curves show more dropout further regulating the network and preventing overfitting.

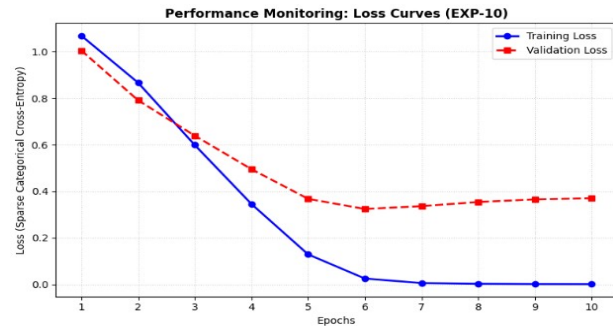


Fig. 10. Smaller batch size of 16 (EXP-10).

As shown in Fig. 10, using a smaller batch size results in a training loss of 0.0008, a validation loss of 0.3702, 100.00% training accuracy, 81.48% validation accuracy, a precision of 0.8333, recall of 0.8148, and an F1-score of 0.8239. The stochastic gradient variance from the setup allows the model to reach an optimal validation loss minimum earlier at epoch 6.

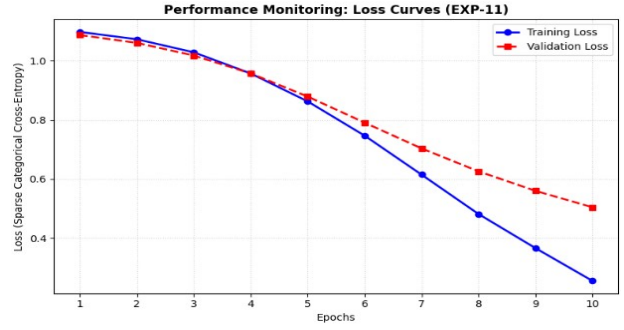


Fig. 11. Larger batch size of 64 (EXP-11).

Illustrated in Fig. 11, the larger batch size results in a training loss of 0.2551, a validation loss of 0.5043, 100.00% training accuracy, 81.48% validation accuracy, a precision of 0.8321, recall of 0.8148, and an F1-score of 0.8234. The graph shows heavily smoothed gradient updates and a stable, continuous loss decrease throughout the 10 epochs.

IV. DISCUSSION & DEBRIEF ANSWERS

A. Authors and Code-Based Optimization vs. No-Code Heuristics (PT-P1 Comparison)

Directly manipulating loss functions, learning rates, and gradient updates in a Python (Keras) environment provides granular control over the network's optimization and learning trajectory compared to the automated heuristics of the PT-P1 no-code platform. The ML for Kids platform abstracts the underlying mathematical architecture to facilitate learning without explicit programming. In contrast, Keras requires defining explicit hyperparameters, such as learning rate schedules and batch sizes, prior to training. The specific trade-off centers on architectural control versus implementation speed. Explicit hyperparameter tuning allows for targeted regularization to prevent rote memorization and close the generalization gap. No-code automation limits the ability to address specific misclassifications or mitigate stochastic gradient variance through exact parameter constraints.

B. Diagnosing Model Fit via Loss Curves & Regularization (Pillar 1 & Pillar 3)

According to the experiment logs, underfitting is identified when both training loss and validation loss remain high, demonstrating the model lacks sufficient capacity or training time to capture underlying data patterns. After establishing baseline convergence in EXP-01 (training loss 0.0028, validation loss 0.3586) and EXP-02 (training loss 0.0327, validation loss 0.4225), underfitting is explicitly observed in EXP-03. Reducing the learning rate to 0.001 yielded a high training loss of 0.9224 and a high validation loss of 0.9576. Conversely, overfitting occurs when the training loss approaches zero while the validation error surges, indicating the network is memorizing training noise. The divergence is visible in EXP-04, where an increased learning rate of 0.01 resulted in a low training loss of 0.0003 but caused the validation loss to climb to 0.5134 after epoch 6.

Introducing a dropout rate randomly deactivates neurons during training to prevent co-adaptation, successfully mitigating overfitting and improving generalization.

Furthermore, adjusting the hidden dim alters the physical capacity of the model. In EXP-05, reducing the hidden dim to 32 restricted network capacity, resulting in a smoother validation loss curve (0.3736) and stabilizing convergence behavior.

C. Data Volume & Diversity vs. Architectural Capacity

The empirical findings demonstrate that adjusting hyperparameters alone is insufficient if the dataset lacks representative diversity. During the PT-P1 phase, scaling the training dataset from 30 to 180 examples significantly improved classification reliability. The dataset expansion increased average confidence for standard inputs from 70.2% to 85.0% and successfully corrected sarcastic anomalies, raising confidence from 61.0% to 80.2%.

Neural network capacity must be strictly balanced with dataset volume because excessive capacity combined with limited data forces the model to memorize the training noise rather than learning generalizable representations. Furthermore, lacking specific contextual representation for untrained inputs, the neural network forcefully assigns unclassified requests to the Negative Complaint category. Testing demonstrated abnormally high confidence scores ranging from 93% to 99% for unrecognized inputs, highlighting a severe operational vulnerability when handling unfamiliar text patterns. Continuous dataset scaling is required to map relationships dynamically without experiencing mathematical exhaustion.

D. Connecting Code Implementation to Slide Architecture (Pipeline Mapping)

The standard neural network training loop maps directly to the actual Python code. The continuous cycle of computing predictions, evaluating errors, and computing gradients backward via the chain rule is triggered simultaneously during the operational training loop. The execution is specifically processed by the `history = model.fit(...)` command within the primary operational loop of the script.

The configuration for evaluating prediction errors and applying optimization algorithms to adjust weights occurs during the optimizer and compilation phase. The `sparse_categorical_crossentropy` loss function and the Adam or SGD optimizer engine are explicitly defined within the `model.compile()` method.

Prior to dense classification, the system extracts features autonomously. The Embedding layer converts raw text tokens into dense numerical vectors within the sequential architecture setup. Immediately following, the `GlobalAveragePooling1D` layer flattens the sequence dimensions. Together, the configured layers perform automated feature extraction directly from the tagged

training data, validating the architecture's capacity to execute without relying on explicit, manually coded logic.

V. CONCLUSION

This study successfully deployed and optimized a sequential neural network for Information Technology Service Management ticket classification. By transitioning to explicit code-based optimization within a Keras environment, the experimental framework gained precise control over mathematical convergence and error navigation. Implementing a systematic One-Factor-At-A-Time strategy provided empirical evidence on how specific hyperparameter modifications directly impact convergence behavior and model generalization.

While the baseline configuration achieved functional accuracy, isolating and tuning specific parameters exposed performance vulnerabilities. Modifying the learning rate heavily influenced the weight update trajectory; reducing the learning rate to 0.001 induced slow, underfitting convergence, while elevating the learning rate to 0.01 triggered pronounced validation loss divergence. Furthermore, adjusting architectural capacity and applying proper regularization proved critical. Expanding the hidden dimension to 128 neurons established superior feature representation, but the structural scaling required a synchronized dropout rate to smooth the validation loss curve and control variance.

Ultimately, the optimal configuration achieved the proper balance of network capacity and regularization, maximizing predictive reliability with an 88.89 percent validation accuracy and a 0.8898 F1-score. The findings confirm that deploying automated triage architectures requires strict programmatic hyperparameter tuning to navigate stochastic gradient updates, mitigate overfitting, and ensure continuous structural stability across enterprise datasets.

REFERENCES

- [1] Abd, Suha & Al-Attar, Bourair & Yousif, Hayder & Al-Rubaie, Abdul & Alzamly, Haider & Sallibi, Anas & Zeeb, Shima & Divekar, Neha & Itankar, Nilisha. (2025). Automated Multimodal Ticket Classification in Public ITSM Systems Using Hierarchical Attention and Deep Neural Networks on Mixed-Format Inputs. 1-7. 10.1109/ICCR67387.2025.11292007.
- [2] S P, Paramesh & K S, Shreedhara. (2022). A DEEP LEARNING BASED IT SERVICE DESK TICKET CLASSIFIER USING CNN. ICTACT Journal on Soft Computing. 13. 2805-2812. 10.21917/ijsc.2022.0399.
- [3] Alsac, Ali & Yılmaz, Ümit & Koçoğlu, Fatma & Seker, Sadi. (2025). Towards Intelligent IT Service Management: A Comparative Evaluation of Machine Learning and Language Models for Ticket Classification and Assignment. 265-270. 10.1109/UBMK67458.2025.11207046.